

UNIVERSITY PARTNER



CHITRAKALA

Digital Art Gallery Platform (A Platform to Showcase, Buy, and Sell Local Artworks)

# Artefact design

Student Name: Nitisha Gyawali

Student Number: 2431163

Award/Course: BSc (Hons) Computer Science

University Email Address: N.Gyawali@wlv.ac.uk

Supervisor: Sachindra Silwal

Reader: Sunita Parajuli

Date of submission: 26 December 2025

## Table of Contents

|                                                                       |    |
|-----------------------------------------------------------------------|----|
| Executive Overview .....                                              | 4  |
| 1. System Context and Purpose .....                                   | 4  |
| SRS Tables for Subsystems .....                                       | 4  |
| 2. Functional Decomposition Diagram (FDD).....                        | 5  |
| User Authentication Module .....                                      | 5  |
| Gallery Management Module.....                                        | 6  |
| Artist Dashboard Module.....                                          | 6  |
| Cart Management Module .....                                          | 7  |
| Payment Processing Module.....                                        | 7  |
| 3. Use Case Diagram: Actor-System Interactions.....                   | 8  |
| 3.1 Purpose and Components.....                                       | 9  |
| 3.2 Use Case Relationships .....                                      | 10 |
| 4. Entity-Relationship Diagram (ERD): Data Structure and Schemas..... | 11 |
| 4.1 Purpose and Data Modeling .....                                   | 11 |
| 4.2 Key Entities and Relationships .....                              | 12 |
| 4.3 Relationship Cardinality .....                                    | 13 |
| 5. Component Diagram: Software Architecture and Dependencies.....     | 14 |
| 5.1 Understanding Component Diagrams.....                             | 14 |
| 5.2 Architectural Layers.....                                         | 14 |
| 6. Sequence Diagrams: Temporal Workflows and Interactions .....       | 16 |
| 6.1 Authentication Sequence.....                                      | 16 |
| 6.2 Gallery Management Sequence.....                                  | 18 |
| 6.3 Artist Upload Sequence .....                                      | 19 |
| 6.4 Payment Sequence.....                                             | 20 |
| 7. Activity Diagrams: Process Flows and Decision Points .....         | 22 |
| 7.1 Authentication Activity Diagram .....                             | 22 |
| Artist Upload Activity Diagram:.....                                  | 23 |
| 7.2 Cart Management Activity Diagram.....                             | 24 |

|                                                 |    |
|-------------------------------------------------|----|
| Cart Update Activity .....                      | 25 |
| 8. Activity Diagram of Payment Management ..... | 26 |
| SRS Table .....                                 | 27 |
| SRS Table for Authentication Management .....   | 27 |
| SRS Table for Gallery Management.....           | 28 |
| SRS Table for Cart Management .....             | 28 |
| SRS Table for Artist Dashboard Management.....  | 29 |
| SRS Table for Payment Management.....           | 30 |
| Wireframe .....                                 | 31 |
| 9. Conclusion .....                             | 34 |

|                                                                       |    |
|-----------------------------------------------------------------------|----|
| Figure 1Functional Decomposition Diagram (FDD) .....                  | 5  |
| Figure 2Use Case Diagram: Actor-System Interactions.....              | 8  |
| Figure 3Entity-Relationship Diagram (ERD): .....                      | 11 |
| Figure 4omponent Diagram: Software Architecture and Dependencies..... | 14 |
| Figure 5Authentication Sequence.....                                  | 16 |
| Figure 6Gallery Management Sequence.....                              | 18 |
| Figure 7Artist Upload Sequence .....                                  | 19 |
| Figure 8Payment Sequence.....                                         | 20 |
| Figure 9Activity Diagrams: Process Flows and Decision Points .....    | 22 |
| Figure 10Artist Upload Activity Diagram: .....                        | 23 |
| Figure 11Cart Management Activity Diagram .....                       | 24 |
| Figure 12Activity Diagram of Payment Management.....                  | 26 |
| Figure 13dashbord wireframe.....                                      | 31 |
| Figure 14login/signing wireframe .....                                | 32 |
| Figure 15userinfo wireframe.....                                      | 32 |
| Figure 16home page .....                                              | 33 |
| Figure 17admin .....                                                  | 33 |

## Executive Overview

Chitrakala is a sophisticated web-based digital art gallery platform engineered to bridge the gap between emerging artists and global art collectors. The system addresses critical pain points in the contemporary art market: limited access to exhibition spaces, prohibitive costs of maintaining physical galleries, and geographic restrictions on audience reach. Built on modern web technologies including React.js with Vite, the platform leverages Context API for state management and localStorage for data persistence, delivering a seamless, role-based experience for artists, collectors, and administrators.

The architectural design documented in this report demonstrates how complex business requirements translate into scalable, maintainable software systems through systematic design methodologies. Each diagram in this report represents a distinct perspective of the system: functional hierarchy, user interactions, data relationships, component dependencies, and behavioral workflows across critical subsystems.

### 1. System Context and Purpose

Chitrakala serves three primary stakeholder groups with distinct but interconnected needs. Artists require a low-friction platform to upload, manage, and monetize their digital creations without intermediaries. Collectors seek an intuitive interface to discover, evaluate, and purchase artwork while maintaining secure transaction records. Administrators oversee system integrity, user management, and content governance to ensure a trustworthy marketplace.

The technical architecture addresses these requirements through a React-based frontend that integrates Context API for centralized state management, eliminating prop-drilling and improving code maintainability. The system implements role-based access control (RBAC), ensuring that sensitive features such as artwork uploads and payment processing are restricted to appropriate user classes. Data persistence leverages browser localStorage, providing offline capability and immediate user experience responsiveness without backend latency.

### SRS Tables for Subsystems

Software Requirements Specification (SRS) tables document functional and non-functional requirements for each subsystem. Each table includes Requirement ID, Description, Type (Functional/Non-Functional), and Priority (High/Medium/Low). These are based on the Chitrakala platform's design, focusing on key features like role-based access, data persistence, and Khalti integration.

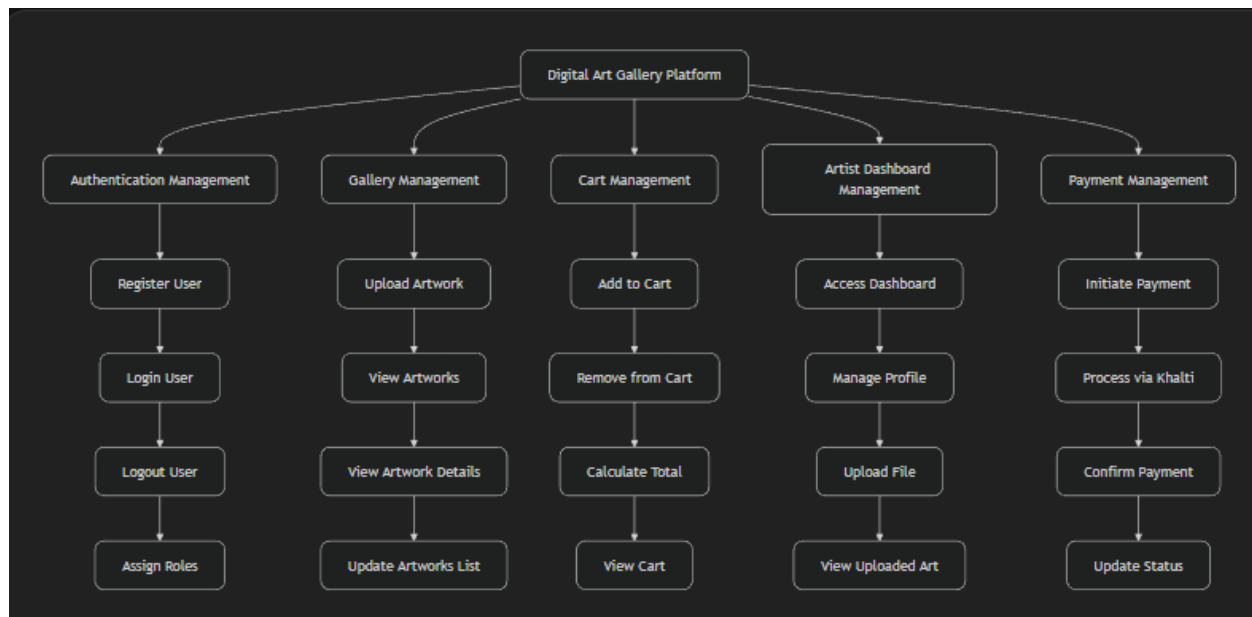
## 2. Functional Decomposition Diagram (FDD)

A Functional Decomposition Diagram is a hierarchical representation that breaks down complex systems into progressively more granular functional components. It operates on the principle of hierarchical decomposition: beginning with the system as a monolithic whole and progressively subdividing it into specialized modules, each responsible for a discrete set of operations. This approach facilitates several critical engineering objectives: it clarifies system scope and boundaries, it distributes complexity across manageable modules, it enables parallel development by different team members, and it provides a reference architecture for testing and quality assurance.

In the context of Chitrakala, the FDD serves as the foundational blueprint, establishing the five primary functional domains that organize all subsequent design decisions.

### 2.2 Detailed Analysis of Functional Modules

Figure 1 Functional Decomposition Diagram (FDD)



### User Authentication Module

The authentication subsystem constitutes the system's security perimeter, controlling access to all protected resources. This module implements three critical security functions: identity verification (confirming user claims through credentials), role assignment (determining whether a user is an Artist, Collector, or Administrator), and session management (maintaining authenticated state without repeated credential entry).

The authentication flow begins with signup, where users provide email and password credentials. These credentials are stored in `localStorage` (in production environments, encrypted backend storage would replace this). Upon login, the system verifies credentials against stored records and generates an authentication token. Subsequent requests include this token, which the system validates before granting access. Role assignment occurs at signup: users select whether they are artists (seeking to upload and sell work) or collectors (seeking to purchase). This role determination unlocks distinct feature sets in downstream modules.

Security implications include password storage strategies (plaintext `localStorage` is acceptable for educational demonstrations; production systems would hash passwords with `bcrypt` or similar algorithms), HTTPS enforcement (preventing credential interception), and session timeout mechanisms (reducing exposure if a device is compromised).

## Gallery Management Module

Gallery Management orchestrates the presentation of artworks to end-users. This module retrieves artwork records from storage, formats them for display, and renders them in an organized, browsable interface. The module implements three operational phases: data retrieval (fetching all artworks from `localStorage` and converting them to component-consumable format), data transformation (organizing artworks into filterable, sortable collections), and presentation (rendering artworks in responsive grid layouts that adapt to screen sizes from mobile devices to desktop displays).

The gallery operates in read-only mode for most users; however, when an artist logs in, the gallery interface subtly transforms to expose creation affordances (buttons to upload new work) and management capabilities (edit and delete options for the artist's own submissions). This context-sensitive interface design reduces visual complexity for users who don't need certain features while maintaining feature discoverability for users who do.

Performance considerations include lazy-loading (loading artwork images only when visible in the viewport), pagination (limiting the number of artworks rendered simultaneously), and image optimization (compressing artwork files without perceptible quality loss). These techniques ensure the gallery remains responsive even as the artwork collection grows to thousands of items.

## Artist Dashboard Module

The Artist Dashboard is a restricted-access area available exclusively to authenticated users with artist roles. This module provides dedicated tools for artwork management, profile customization, and business analytics. Artists can upload new artworks (selecting image files and entering metadata such as title, description, price, and medium), view their upload history

(with metrics on views, cart additions, and purchases), edit artwork details (updating prices or descriptions), and delete underperforming works.

The upload mechanism implements file encoding: when an artist selects an image, the browser converts it to Base64 format (a text representation of binary data suitable for storage in JSON and localStorage). Upon upload, the system appends a unique identifier (typically a timestamp or UUID), title, artist name, description, price, and encoded image data to the artwork collection. The system immediately reflects this new artwork in the gallery, available for discovery and purchase.

Dashboard insights might include artwork performance metrics: how many times each artwork has been viewed, how many times it appears in collector carts, and how many times it has been purchased. These metrics inform artistic and commercial decisions: if certain styles attract higher engagement, artists can create more work in those styles.

## Cart Management Module

Cart Management implements a temporary holding area where collectors assemble purchases before committing to payment. This module provides add-to-cart functionality (appending artworks to a collector-specific cart), remove-from-cart functionality (deleting unwanted items), quantity management (allowing collectors to purchase multiple copies of digital works), and total price calculation (summing the prices of all items in the cart, potentially applying discount codes or promotional pricing).

The cart persists in localStorage under the collector's user session, ensuring that items remain in the cart even if the browser is closed and later reopened. This persistence improves user experience by preventing loss of selections due to accidental navigation or application crashes.

The cart system implements price accuracy mechanisms: before proceeding to checkout, the system verifies that all artworks in the cart are still available at their listed prices. If an artwork has been deleted by its artist or its price has changed, the system alerts the collector and provides options to remove the artwork or proceed anyway. This prevents surprise charges at checkout.

## Payment Processing Module

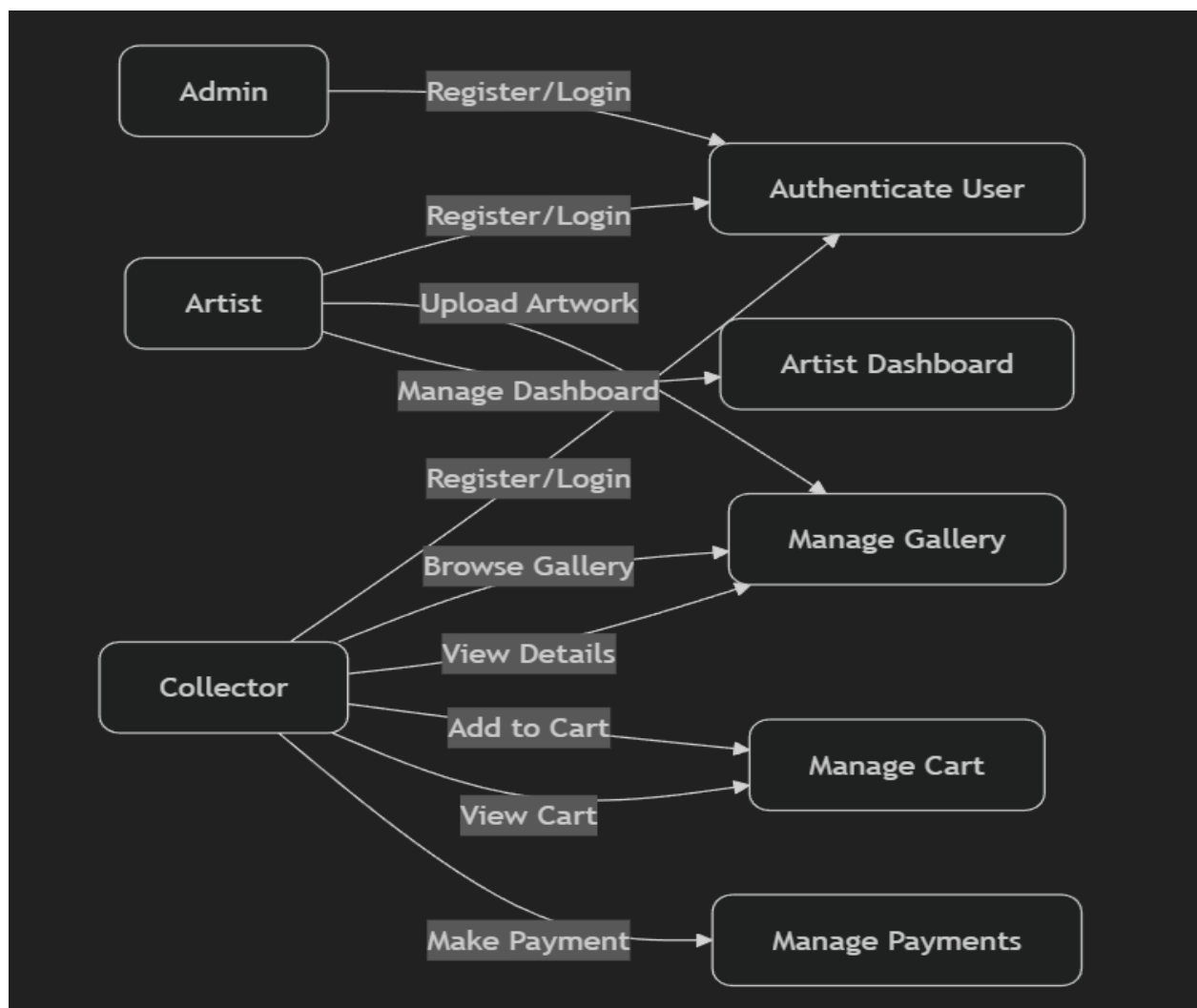
The Payment Processing module orchestrates financial transactions. The current implementation includes placeholder logic for Khalti, a popular digital payment gateway in South Asia, accepting mobile payments, bank transfers, and e-wallet transactions. In the demonstration environment, payment processing is simulated: collectors proceed through a checkout flow, and their payment status is marked as "completed" in localStorage.

Production implementation would integrate Khalti's API, sending purchase details to Khalti's servers for payment processing. Khalti returns a success or failure status, which the system records in an Order entity (creating a permanent record of the transaction). Order records include timestamp, collector ID, list of purchased artworks, total amount paid, and payment status—essential for dispute resolution, refund processing, and tax compliance.

Security considerations include PCI DSS compliance (standards for payment card processing, though mitigated by delegating to Khalti rather than processing cards directly), SSL/TLS encryption (ensuring payment data travels over encrypted channels), and fraud detection (identifying suspicious patterns like repeat failed payment attempts).

### 3. Use Case Diagram: Actor-System Interactions

Figure 2 Use Case Diagram: Actor-System Interactions





### 3.1 Purpose and Components

A Use Case Diagram models interactions between external actors (users, other systems) and the software system. It identifies what actions the system can perform, who can perform them, and what preconditions and outcomes characterize each interaction. Unlike the FDD which emphasizes system structure, the Use Case Diagram emphasizes system behavior and user goals.

In the Chitrakala diagram, three primary actors interact with the system:

**Artist Actor:** Represents users with creative content and commerce objectives. The artist's primary goal is to reach potential collectors without geographic or intermediary restrictions. Use cases accessible to artists include "Authenticate," "Upload Artwork," "Manage Uploaded Works," and "View Sales Analytics." Each use case represents a discrete goal-oriented interaction: uploading is goal (make artwork available for sale), managing uploaded works is goal (maintain the collection, respond to collector feedback), and analytics is goal (understand commercial performance).

**Collector Actor:** Represents users with purchasing intent. The collector seeks to discover, evaluate, and acquire artwork matching aesthetic preferences and budget. Use cases include "Authenticate," "Browse Gallery," "View Artwork Details," "Add to Cart," and "Checkout." These use cases form a natural sequence: a collector authenticates (establishing identity and enabling personalized features), browses the gallery (discovering possibilities), views details of interesting artworks (learning about artistic intent and medium), adds selections to the cart (committing to consideration without final purchase), and checks out (initiating payment).

**Admin Actor:** Represents system operators with governance responsibilities. The current design documents admin as a future role; however, comprehensive designs would include use cases such as "Manage User Accounts," "Moderate Content," "Process Disputes," and "Generate Reports." Moderation is critical to prevent inappropriate content; dispute management handles refund requests or fraud complaints; reporting supports business intelligence and compliance audits.

## 3.2 Use Case Relationships

Standard notation distinguishes several use case relationships:

**Includes Relationship:** Indicates that one use case mandatorily includes another as a subtask. For example, "Checkout" includes "Process Payment" because payment is an integral step within checkout, not a standalone option.

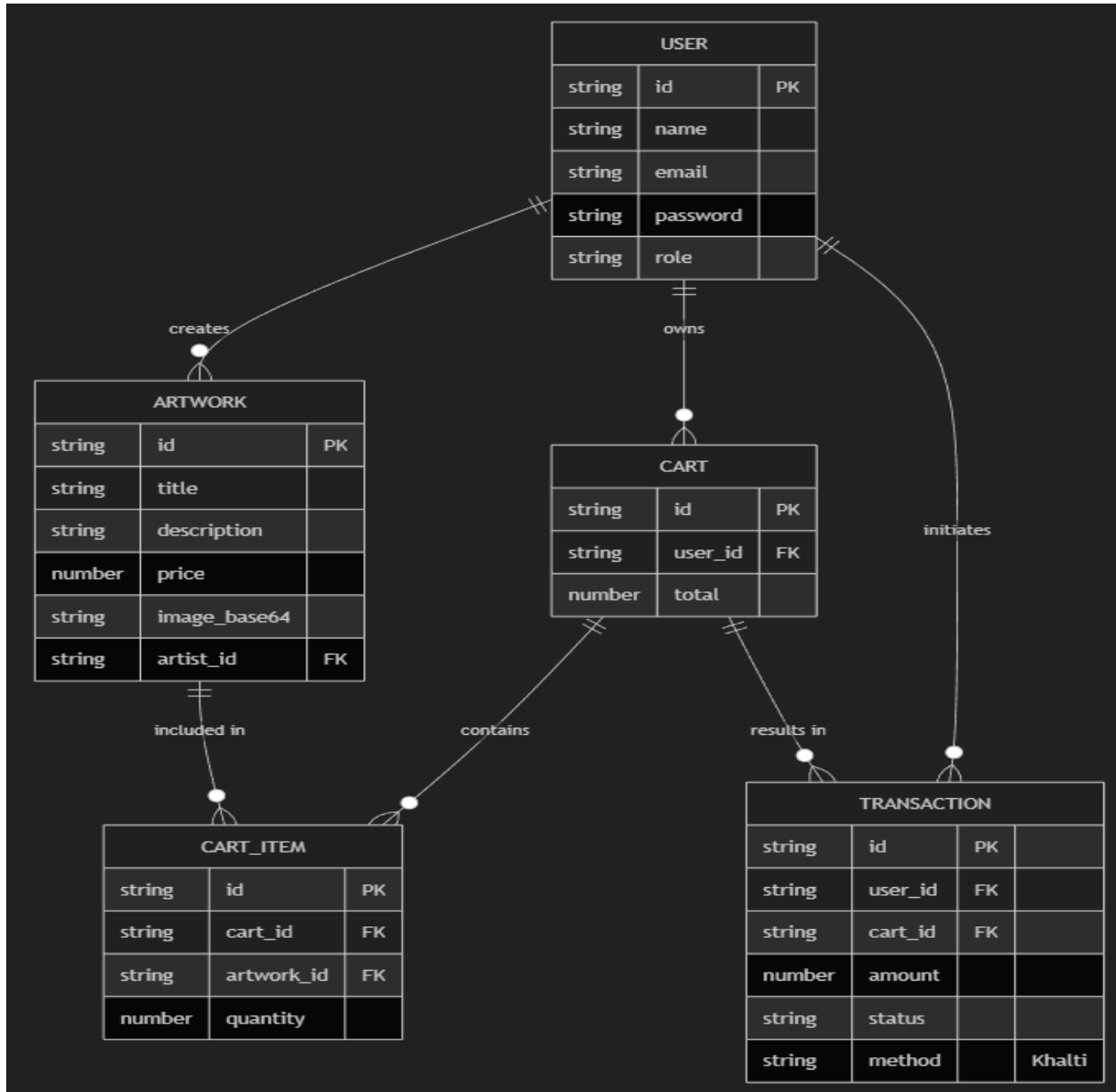
**Extends Relationship:** Indicates that one use case optionally adds behavior to another. For example, "View Artwork Details" might extend with "View Artist Profile," allowing interested collectors to learn about an artist's background, education, or other works.

**Inheritance:** Indicates that an actor can fulfill multiple roles. A user might authenticate as both artist and collector, inheriting use cases from both roles (uploading as an artist on Tuesday, purchasing as a collector on Wednesday).

In Chitrakala's design, the most critical relationships are inclusivity (checkout includes payment), extension (browsing can extend to filtering by price or style), and role-based accessibility (upload is only available to artists, protecting data integrity).

## 4. Entity-Relationship Diagram (ERD): Data Structure and Schemas

Figure 3 Entity-Relationship Diagram (ERD):



### 4.1 Purpose and Data Modeling

An Entity-Relationship Diagram defines the persistent data structures that the system maintains and the relationships between those structures. Each entity represents a noun-like concept (User, Artwork, Cart, Order); each relationship represents a verb-like association (User uploads

Artwork, Collector adds Artwork to Cart). The ERD serves as the blueprint for database schema design and data validation logic.

## 4.2 Key Entities and Relationships

### User Entity

The User entity represents both artists and collectors within a unified structure, with a role attribute distinguishing between them. Core attributes include:

- `user_id`: Unique identifier (typically a UUID or database-assigned integer)
- `email`: Unique email address for login and communication
- `password`: Hashed password (bcrypt or similar in production)
- `role`: Enumeration ("artist" or "collector")
- `username`: Display name for profiles
- `profile_picture`: Optional image representing the user
- `bio`: Text describing the user (artists: artistic philosophy, collectors: collecting interests)
- `joined_date`: Timestamp of account creation

### Artwork Entity

The Artwork entity represents individual digital creations. Attributes include:

- `artwork_id`: Unique identifier
- `artist_id`: Foreign key referencing the User who created it
- `title`: Name of the artwork
- `description`: Detailed explanation of artistic intent, medium, and inspiration
- `image_data`: Base64-encoded image or URI referencing an image storage service
- `price`: Numerical price in the local currency
- `medium`: Medium category (digital painting, 3D render, photography, etc.)
- `dimensions`: Width and height in pixels
- `created_date`: Timestamp of creation
- `upload_date`: Timestamp of upload to Chitrakala

## Cart Entity

The Cart entity represents a collector's shopping session. Attributes include:

- `cart_id`: Unique identifier
- `collector_id`: Foreign key referencing the User (Collector role)
- `artworks`: List of `artwork_ids` and quantities
- `total_price`: Calculated sum of all items
- `created_date`: Session start time
- `last_modified_date`: Last update time

## Order Entity

The Order entity represents a completed transaction. Attributes include:

- `order_id`: Unique identifier
- `collector_id`: Foreign key referencing the User who made the purchase
- `artworks`: List of purchased `artwork_ids` and quantities
- `total_amount`: Amount paid
- `payment_status`: Enumeration ("pending," "completed," "failed," "refunded")
- `payment_method`: Enumeration ("khalti," "bank\_transfer," "card," etc.)
- `order_date`: Timestamp of purchase
- `delivery_date`: Timestamp when digital files were delivered or made available

## 4.3 Relationship Cardinality

Cardinality expresses how many instances of one entity relate to instances of another:

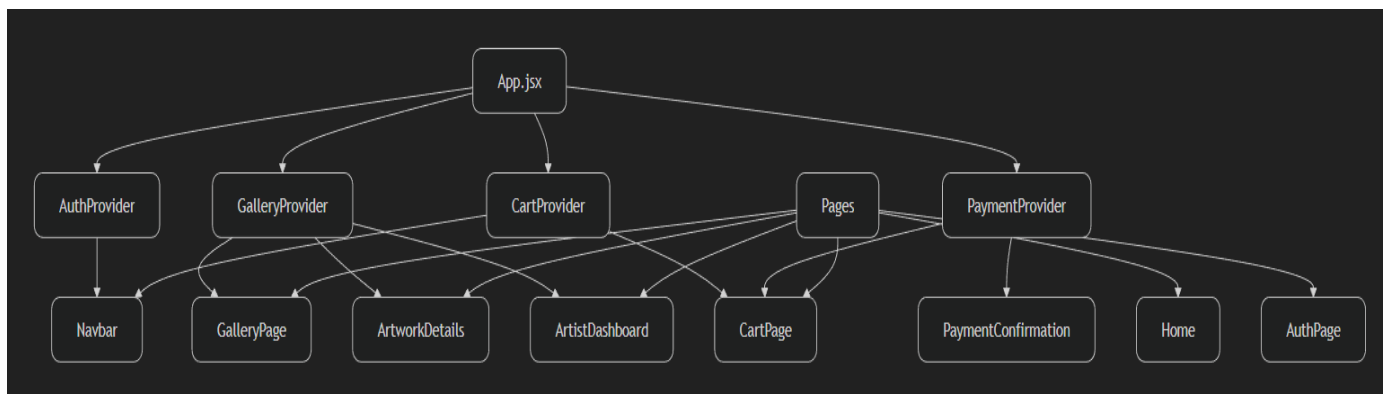
- User to Artwork : One user (artist) uploads many artworks; one artwork belongs to exactly one user. This is a one-to-many relationship.
- Collector to Cart : One collector has exactly one active shopping cart at any time. (Upon checkout, the cart is archived and a new one is created.)
- Cart to Artwork : One cart contains many artworks; one artwork can appear in many collectors' carts simultaneously.

- Collector to Order : One collector places many orders; one order belongs to exactly one collector.

These relationships ensure data integrity: if an artist is deleted, cascading rules determine whether their artworks are deleted or reassigned; if an artwork is deleted, carts and orders containing that artwork are updated to reflect unavailability.

## 5. Component Diagram: Software Architecture and Dependencies

Figure 4 component Diagram: Software Architecture and Dependencies



### 5.1 Understanding Component Diagrams

A Component Diagram reveals the modular structure of the software itself—how the application is organized into reusable, independently testable pieces. Components are coarser-grained than classes but finer-grained than subsystems. In React applications, components are functions that return user interface elements; in enterprise systems, components are modules, libraries, or microservices.

### 5.2 Architectural Layers

#### Context Providers (State Management)

The Chitrakala architecture centralizes state in three Context providers:

- **AuthContext:** Manages authentication state (current user, authentication status, login/logout functions). All components that need to check whether a user is logged in or determine their role consume AuthContext.

- **GalleryContext:** Manages artwork collection state (all available artworks, add/edit/delete artwork functions). Gallery and ArtworkDetail components consume this context to display and update artworks.
- **CartContext:** Manages shopping cart state (items in cart, add/remove item functions, total price calculations). Cart and Checkout components consume this context.

## Page Components

Page components represent top-level screens:

- **Home:** Landing page with call-to-action, featured artists, featured artworks, and navigation
- **Gallery:** Searchable, filterable gallery of all artworks with grid layout
- **ArtworkDetails:** Detailed view of a single artwork, including description, artist profile link, price, and "Add to Cart" button
- **ArtistDashboard:** Restricted-access dashboard for uploading artworks, editing artist profile, and viewing performance analytics
- **Cart:** Shopping cart display with items, prices, remove buttons, and checkout button
- **Checkout:** Final purchase confirmation with payment method selection and "Complete Purchase" button

## Shared Components

Reusable components improve maintainability by centralizing functionality:

- **Header:** Navigation bar with branding, search, user menu, and cart icon
- **ArtworkCard:** Displays a single artwork in grid layout (title, thumbnail, price, artist name, add-to-cart button)
- **UserProfile:** Displays user information with edit capability if viewing own profile
- **PaymentForm:** Collects payment method selection and processes payment

## Integration Points

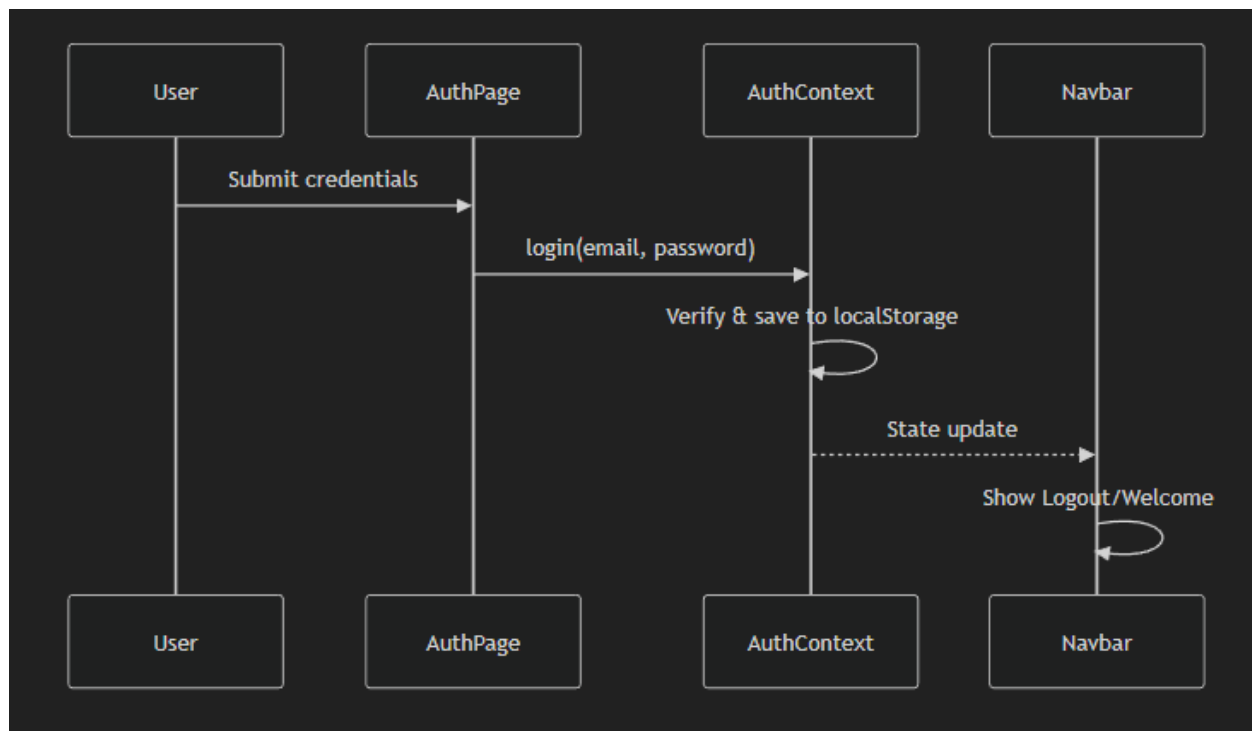
Components integrate through context: when a collector clicks "Add to Cart" on an ArtworkCard, the ArtworkCard dispatches an action to CartContext, which updates cart state. The Cart component listens to CartContext and automatically re-renders to reflect the new item. This event-driven architecture decouples components: ArtworkCard doesn't need to know or care how the cart component handles updates.

## 6. Sequence Diagrams: Temporal Workflows and Interactions

### 6.1 Authentication Sequence

The Authentication Sequence Diagram models the temporal flow of signup and login processes. A sequence diagram includes actors (on the horizontal axis), components (swim lanes), and messages (arrows indicating function calls).

Figure 5 Authentication Sequence



#### Signup Sequence:

1. User enters email, password, role selection, and submits signup form
2. SignupPage component receives form submission event
3. SignupPage calls signup() function in AuthContext
4. AuthContext validates email (not already registered) and password (meets security requirements)
5. AuthContext stores user record in localStorage
6. AuthContext sets currentUser state and returns success



7. SignupPage redirects to Home or Dashboard depending on role
8. Header component observes AuthContext update and displays user name and logout button

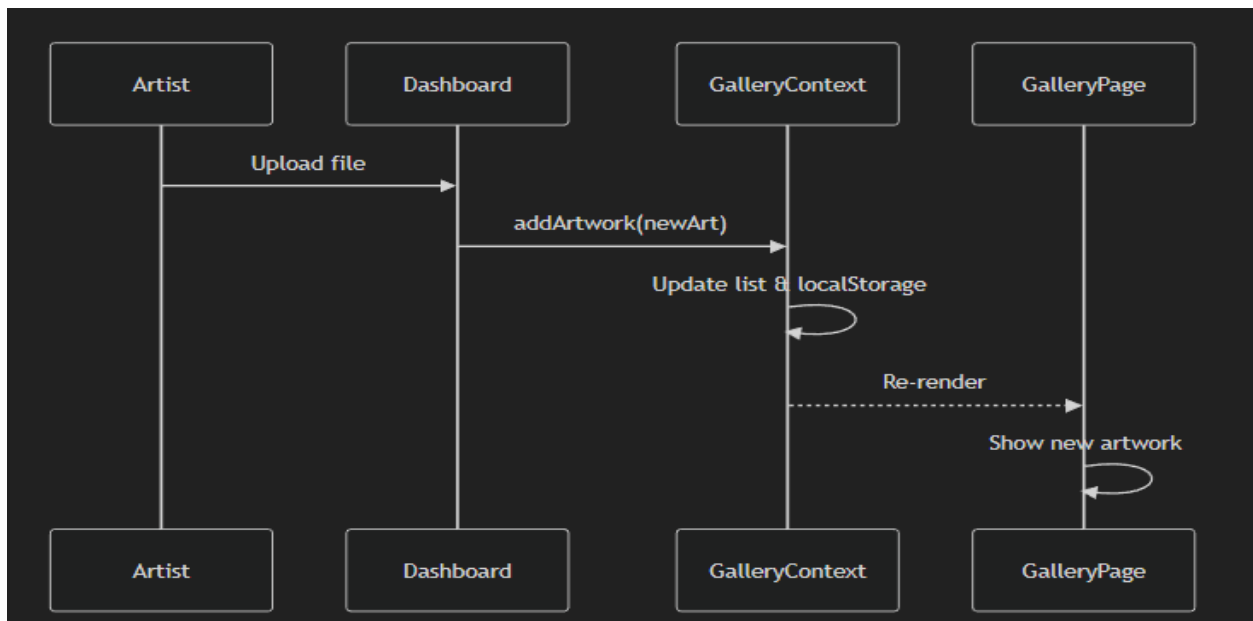
**Login Sequence:**

1. User enters email and password on LoginPage
2. LoginPage calls login() function in AuthContext
3. AuthContext retrieves user record from localStorage matching email
4. AuthContext compares submitted password against stored password (in production, using bcrypt comparison)
5. If match, AuthContext sets currentUser state
6. If mismatch, AuthContext returns error message
7. Upon successful login, application navigates to Home (collectors) or ArtistDashboard (artists)

The sequence diagram's power is illustrating temporal dependencies: signup must complete before login is possible; login must complete before accessing restricted features. It also reveals potential race conditions: if two signup requests for the same email arrive simultaneously, the system should handle this gracefully (the second request should fail with "email already registered").

## 6.2 Gallery Management Sequence

Figure 6 Gallery Management Sequence



### Gallery Loading Sequence:

1. Gallery page component mounts
2. Gallery component calls `loadArtworks()` from `GalleryContext`
3. `GalleryContext` retrieves artwork array from `localStorage`
4. `GalleryContext` transforms artworks (calculating derived fields like "days since upload") and updates state
5. Gallery component receives updated state and re-renders
6. Browser renders `ArtworkCard` components in a grid layout
7. User views gallery and considers artworks

### Add to Cart Sequence:

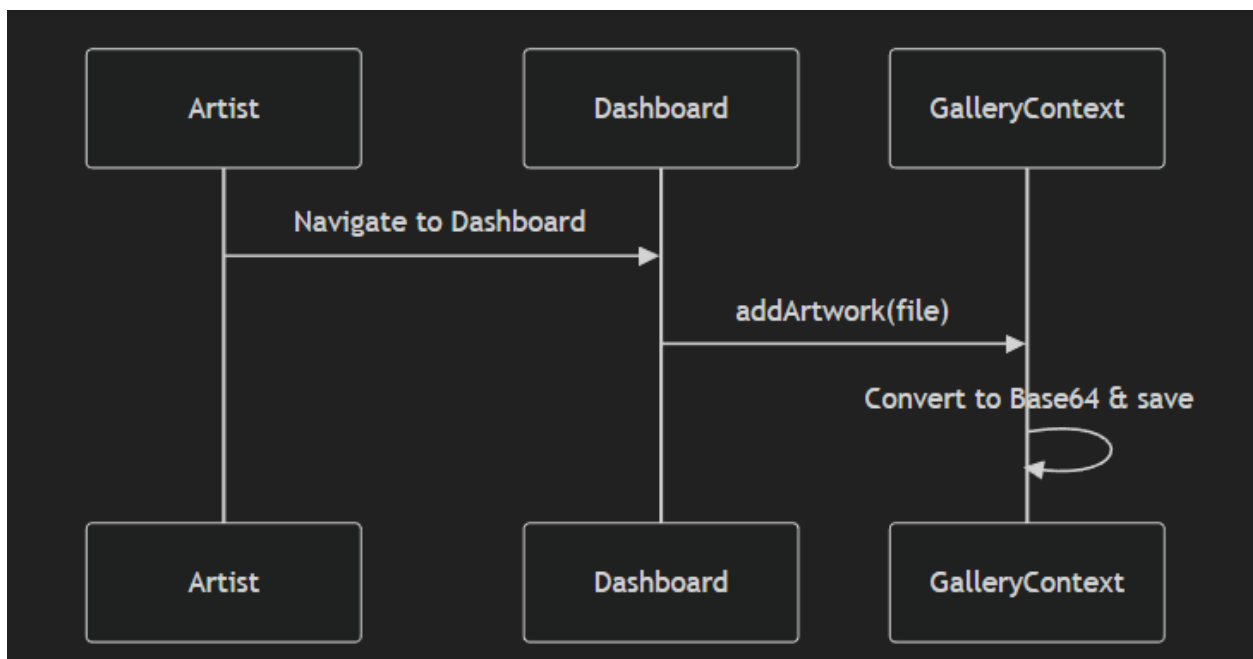
1. Collector clicks "Add to Cart" button on an `ArtworkCard`
2. `ArtworkCard` dispatches `addToCart(artwork_id)` action to `CartContext`
3. `CartContext` retrieves the artwork from `GalleryContext` to get current price
4. `CartContext` adds artwork to cart array and recalculates total
5. `CartContext` updates `localStorage` with new cart state

6. Header component (listening to CartContext) updates cart icon to display new item count
7. Cart page (if open) re-renders to reflect new item
8. Optional: Toast notification appears confirming addition to cart

This sequence reveals a critical design decision: cart operations must retrieve current artwork prices from GalleryContext (rather than storing stale prices in the cart) to prevent collectors from purchasing at outdated prices.

## 6.3 Artist Upload Sequence

Figure 7 Artist Upload Sequence



### Artwork Upload Sequence:

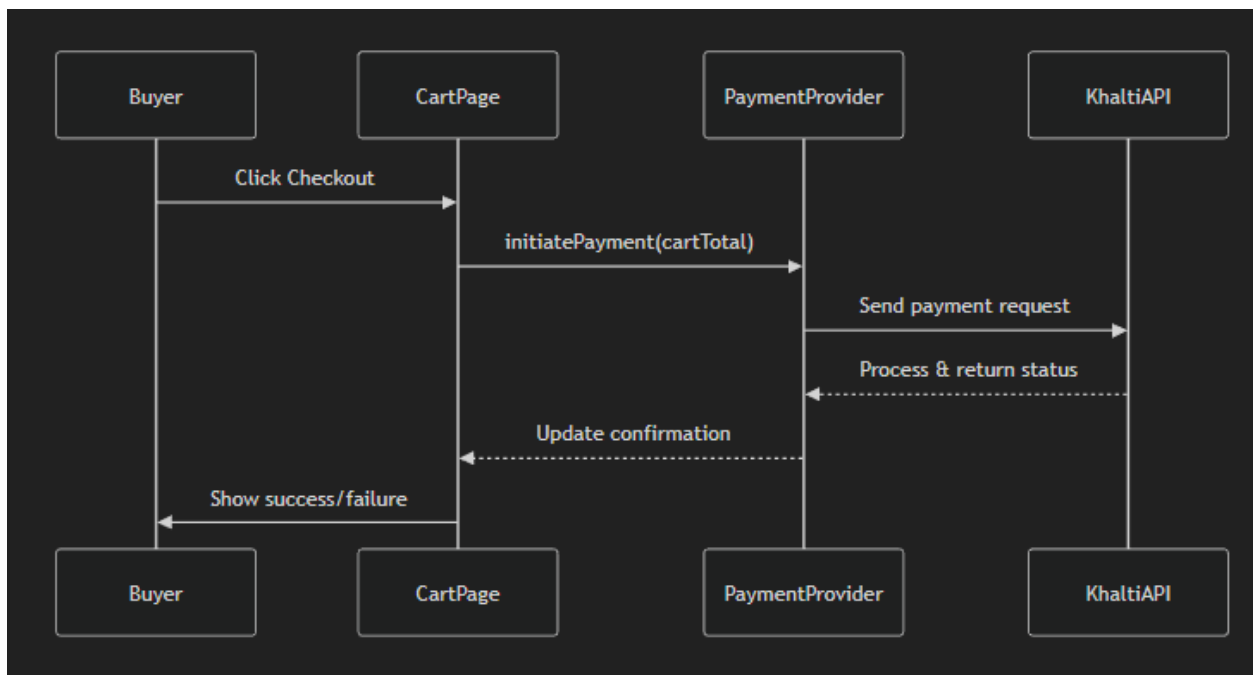
1. Artist navigates to Dashboard (authenticated with artist role)
2. Dashboard displays upload form
3. Artist selects image file (browser's file picker opens)
4. Artist enters title, description, price, medium
5. Artist clicks "Upload" button
6. Dashboard reads file using FileReader API and converts to Base64
7. Dashboard dispatches uploadArtwork(metadata, imageBase64) to GalleryContext

8. GalleryContext creates new artwork object with unique ID, timestamp, and artist ID
9. GalleryContext appends artwork to artworks array in localStorage
10. GalleryContext updates state and notifies all consumers
11. Dashboard displays success message and clears form
12. Gallery page (if open) re-renders to show new artwork at top of grid

This sequence demonstrates asynchronous file handling: reading a file from the user's device is asynchronous (the browser doesn't block while reading), so FileReader uses callbacks to signal completion.

## 6.4 Payment Sequence

Figure 8Payment Sequence



### Checkout Sequence:

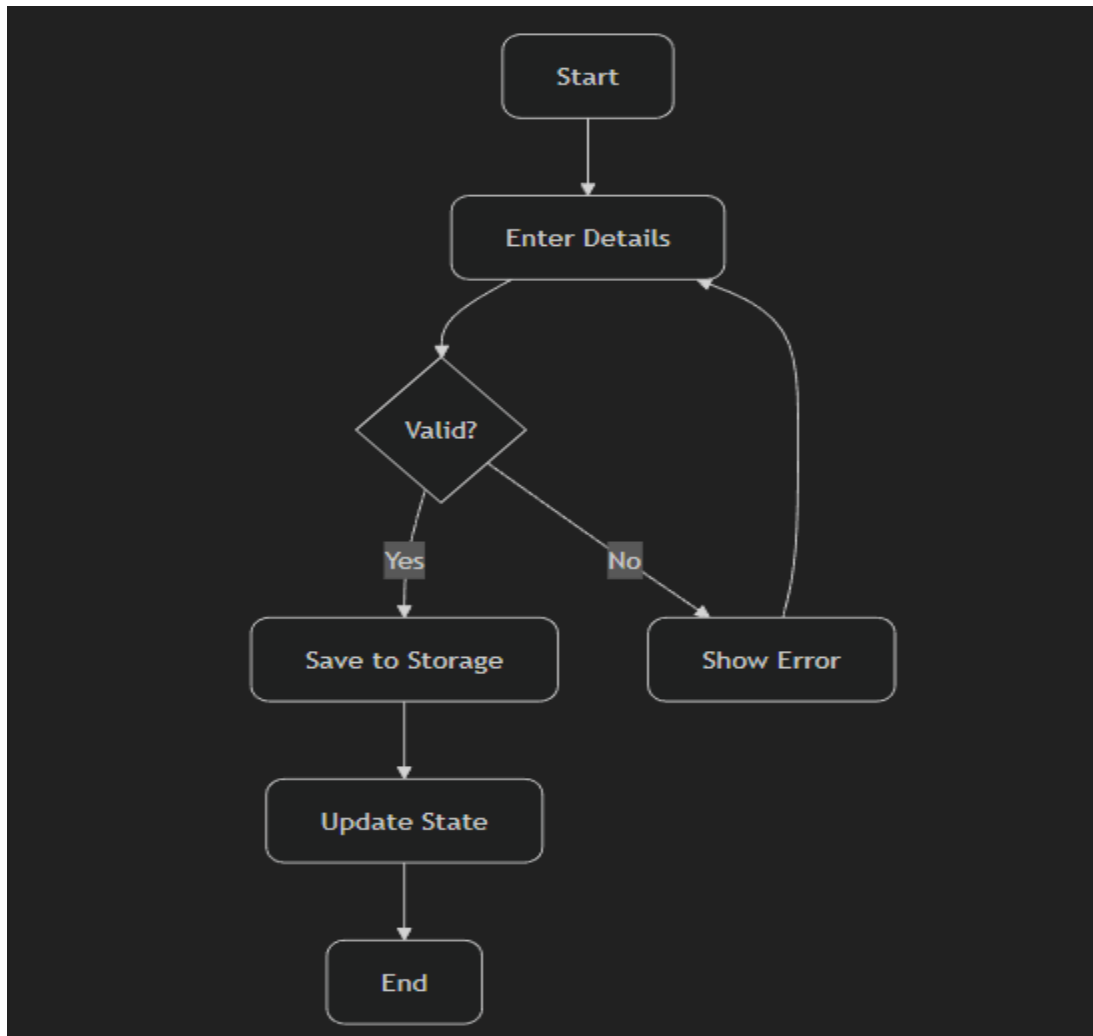
1. Collector clicks "Checkout" button from Cart page
2. Checkout page displays order summary (items, quantities, prices, total)
3. Checkout page displays payment method options (Khalti, bank transfer, etc.)
4. Collector selects payment method and confirms

5. Checkout page calls `processPayment(order_details, payment_method)` from `PaymentContext`
6. `PaymentContext` initiates Khalti API request with order details (in production; simulated locally for this demo)
7. `PaymentContext` waits for Khalti response
8. (In simulation, `PaymentContext` immediately generates success response)
9. `PaymentContext` creates Order entity with `order_id`, `collector_id`, artworks list, `total_amount`, and `payment_status="completed"`
10. `PaymentContext` stores order in orders array in `localStorage`
11. `PaymentContext` clears collector's cart
12. Checkout page displays order confirmation with order number and instructions for accessing purchased artworks
13. Collector's email receives order confirmation (in production; simulated by displaying on-screen in this demo)

The simulation accepts that real payment involves external systems; the diagram shows integration points where Khalti would be called.

## 7. Activity Diagrams: Process Flows and Decision Points

Figure 9 Activity Diagrams: Process Flows and Decision Points

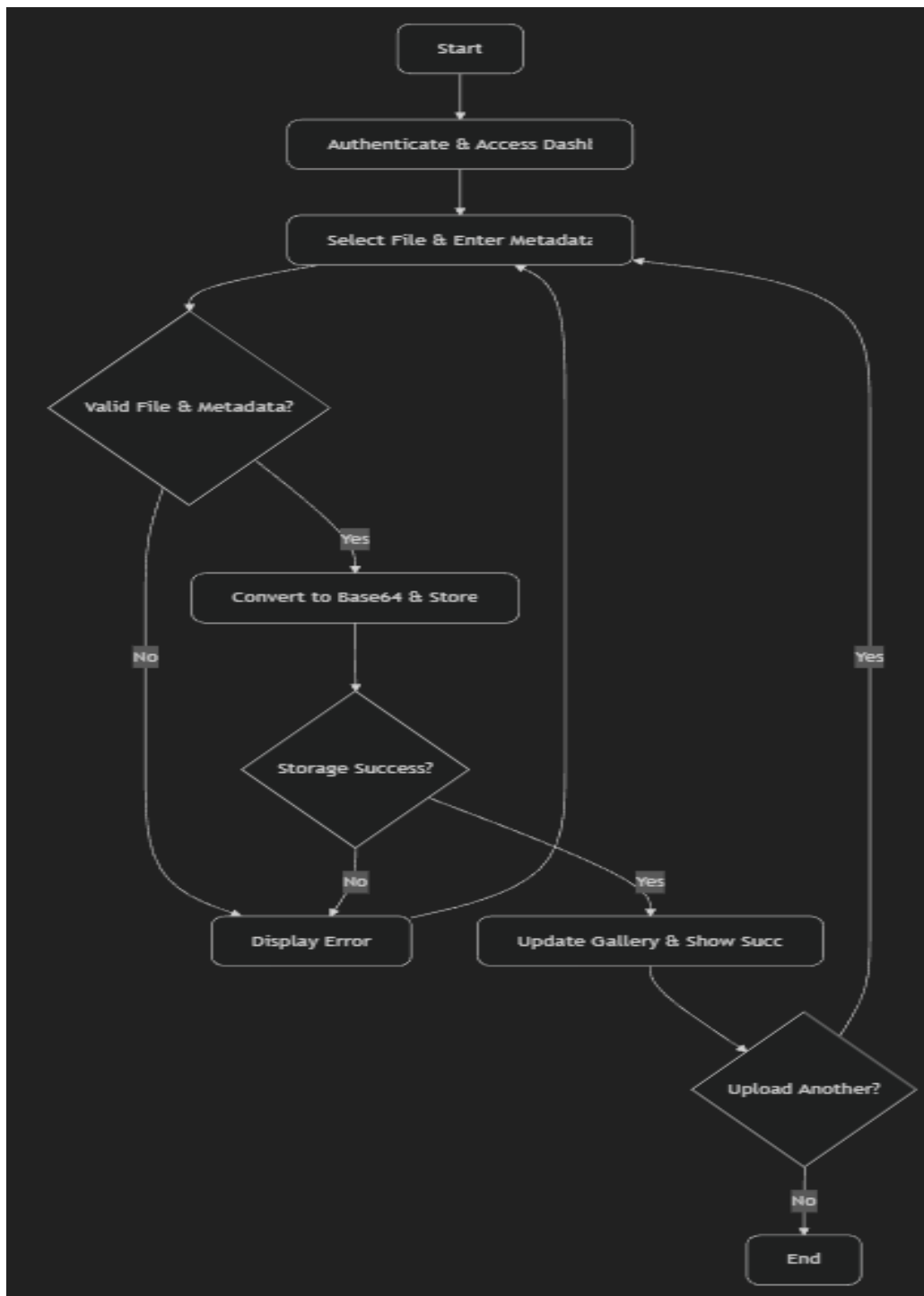


### 7.1 Authentication Activity Diagram

Activity diagrams model processes as sequences of activities, decision points (diamonds), and forks/joins (parallel processing).

## Artist Upload Activity Diagram:

Figure 10Artist Upload Activity Diagram:



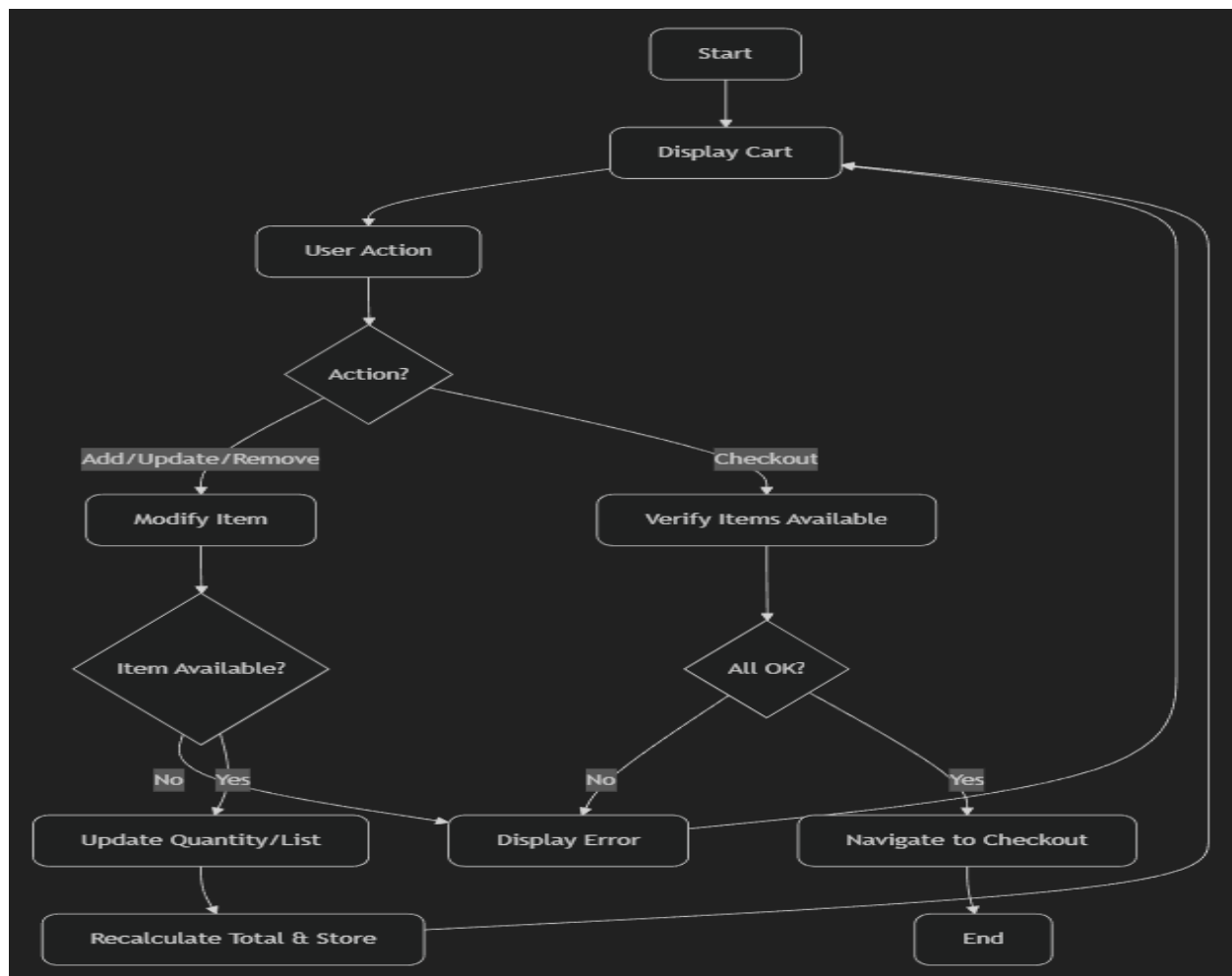
START → Authenticate as Artist → Access Dashboard → Upload Form Loads → [Wait for User] → User Selects File → [File Selected?] → (If No: Return to File Selector) → (If Yes: Read File) → File Read Complete → Convert to Base64 → [Conversion Success?] → (If No: Display Error, Return to

Upload Form) → (If Yes: Create Artwork Object) → Validate Metadata (title, price, medium) → [Validation Passes?] → (If No: Display Validation Errors, Return to Form) → (If Yes: Store in localStorage) → [Storage Success?] → (If No: Display Storage Error) → (If Yes: Update Gallery Display) → Display Success Message → [User Uploads Another?] → (If Yes: Return to Upload Form) → (If No: Navigate to Gallery) → END

Decision points (diamonds) represent conditional branches: the process flow depends on the result of a condition. For example, "File Selected?" branches into two paths: if yes, proceed to file reading; if no, return to file selection. This structure makes the control flow explicit and testable.

## 7.2 Cart Management Activity Diagram

Figure 11 Cart Management Activity Diagram





## Cart Update Activity

START → Display Current Cart Items → [Display Format: Item Card with Quantity and Price] → User Actions [Add Item / Remove Item / Update Quantity / Proceed to Checkout] → [Action Type?] →

(If Add Item) → Receive Item ID → Retrieve Item Details from Gallery → [Item Available?] → (If No: Display "Item No Longer Available") → (If Yes: Add to Cart Items List) → Recalculate Total Price → Update localStorage → Refresh Cart Display → Return to Cart View

(If Remove Item) → Receive Item ID → Find Item in Cart → Remove from Items List → Recalculate Total Price → Update localStorage → Refresh Cart Display → Return to Cart View

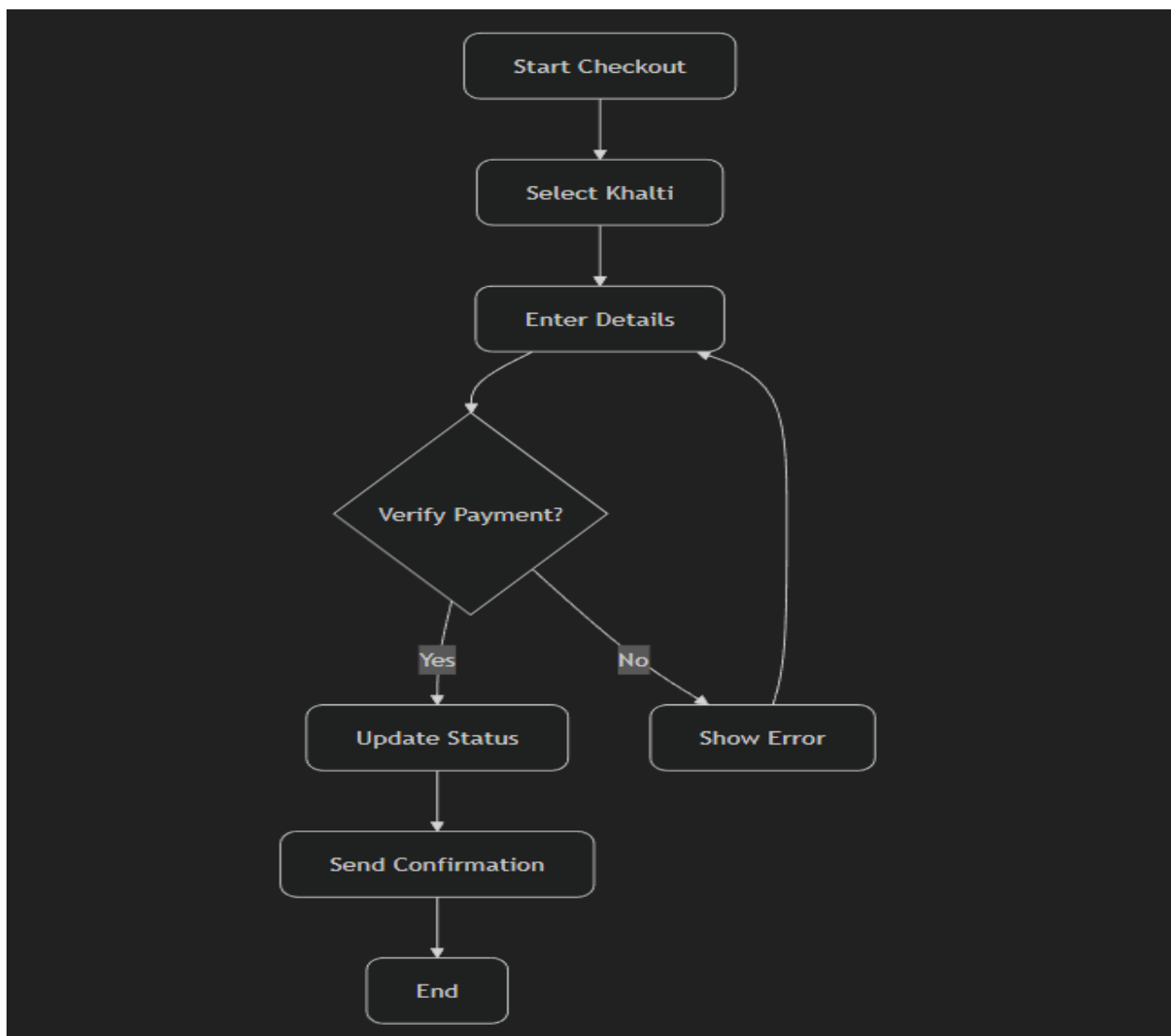
(If Update Quantity) → Receive Item ID and New Quantity → [New Quantity > 0?] → (If No: Remove Item from Cart) → (If Yes: Update Item Quantity) → Recalculate Total Price → Update localStorage → Refresh Cart Display → Return to Cart View

(If Proceed to Checkout) → Verify All Items Still Available → [All Items Available?] → (If No: Display Warning, Show Unavailable Items) → (If Yes: Create Order Summary) → Navigate to Checkout Page → END

The activity diagram's value is revealing sequential dependencies: total price must be recalculated after any addition, removal, or quantity change; cart state must persist to localStorage to survive browser restarts; the system must verify item availability before checkout to prevent purchasing items that were deleted by artists.

## 8. Activity Diagram of Payment Management

Figure 12 Activity Diagram of Payment Management



This activity diagram outlines the Khalti payment process in the Chitrakala platform. The flow begins when the user initiates checkout from the cart. The user then selects Khalti as the preferred payment method, followed by entering necessary payment details (such as mobile number, PIN, or OTP as required by Khalti). After submission, the system reaches a critical decision point to verify the payment through Khalti's gateway. If the verification succeeds ("Yes"), the transaction status is updated in the system (e.g., marking the order as paid), a confirmation message or receipt is sent to the user, and the process concludes successfully. If verification fails ("No"), an error message is displayed to the user, allowing them to retry entering details or cancel the transaction. The loop back on failure ensures user-friendly error

handling, while the success path provides clear closure, making the payment experience secure, straightforward, and aligned with local Nepali digital wallet practices.

## SRS Table

### SRS Table for Authentication Management

| Requirement ID | Description                                                                  | Type           | Priority |
|----------------|------------------------------------------------------------------------------|----------------|----------|
| AUTH-001       | Users must register with email, password, and role (artist/collector/admin). | Functional     | High     |
| AUTH-002       | System verifies credentials for login and assigns role-based access.         | Functional     | High     |
| AUTH-003       | Logout clears session and localStorage data.                                 | Functional     | Medium   |
| AUTH-004       | Role assignment restricts access (e.g., dashboard for artists only).         | Functional     | High     |
| AUTH-005       | Input validation for forms (e.g., email format, password strength).          | Non-Functional | Medium   |
| AUTH-006       | Persistence using localStorage for session continuity.                       | Non-Functional | High     |
| AUTH-007       | Error handling for invalid credentials or duplicate emails.                  | Non-Functional | Medium   |

This SRS focuses on secure user access and role management, essential for the platform's RBAC system.

## SRS Table for Gallery Management

| Requirement ID | Description                                                    | Type           | Priority |
|----------------|----------------------------------------------------------------|----------------|----------|
| GALL-001       | Display artworks in responsive grid format from localStorage.  | Functional     | High     |
| GALL-002       | View detailed artwork info (title, price, description, image). | Functional     | High     |
| GALL-003       | Dynamic updates on new uploads without page reload.            | Functional     | Medium   |
| GALL-004       | Search/filter by category, price, or location.                 | Functional     | Medium   |
| GALL-005       | Responsive design for mobile/desktop viewing.                  | Non-Functional | High     |
| GALL-006       | Base64 encoding for image handling and display.                | Non-Functional | Medium   |
| GALL-007       | Error handling for missing or corrupted artwork data.          | Non-Functional | Low      |

Explanation: Requirements emphasize dynamic content display and user-friendly browsing for art discovery.

## SRS Table for Cart Management

| Requirement ID | Description                                  | Type           | Priority |
|----------------|----------------------------------------------|----------------|----------|
| CART-001       | Add selected artworks to cart with quantity. | Functional     | High     |
| CART-002       | Remove items or update quantities in cart.   | Functional     | High     |
| CART-003       | Calculate and display real-time total price. | Functional     | High     |
| CART-004       | View cart summary with item details.         | Functional     | Medium   |
| CART-005       | Persistence of cart data in localStorage.    | Non-Functional | High     |

|          |                                                     |                |        |
|----------|-----------------------------------------------------|----------------|--------|
| CART-006 | Validation for item availability before adding.     | Non-Functional | Medium |
| CART-007 | Error messages for empty cart or unavailable items. | Non-Functional | Low    |

Explanation: This SRS supports e-commerce functionality with persistent, accurate shopping cart operations.

SRS Table for Artist Dashboard Management

| Requirement ID | Description                                                     | Type           | Priority |
|----------------|-----------------------------------------------------------------|----------------|----------|
| DASH-001       | Role-restricted access for artists only.                        | Functional     | High     |
| DASH-002       | Upload artwork files with metadata (title, price, description). | Functional     | High     |
| DASH-003       | Convert uploads to Base64 and store in localStorage.            | Functional     | High     |
| DASH-004       | View and manage uploaded artworks list.                         | Functional     | Medium   |
| DASH-005       | Profile management (update name, bio).                          | Functional     | Medium   |
| DASH-006       | Responsive UI for file selection and preview.                   | Non-Functional | High     |
| DASH-007       | Validation and error handling for uploads.                      | Non-Functional | Medium   |

Explanation: Focuses on empowering artists with secure, easy content management tools.

SRS Table for Payment Management

| Requirement ID | Description                                           | Type           | Priority |
|----------------|-------------------------------------------------------|----------------|----------|
| PAY-001        | Initiate checkout from cart with Khalti selection.    | Functional     | High     |
| PAY-002        | Process payment via Khalti API (mobile/PIN/OTP).      | Functional     | High     |
| PAY-003        | Verify transaction and update order status.           | Functional     | High     |
| PAY-004        | Send confirmation email/receipt on success.           | Functional     | Medium   |
| PAY-005        | Secure handling of payment details (no storage).      | Non-Functional | High     |
| PAY-006        | Error handling and retry for failed verifications.    | Non-Functional | High     |
| PAY-007        | Integration with localStorage for status persistence. | Non-Functional | Medium   |

Explanation: Ensures secure, reliable local payment processing aligned with Nepali digital wallets.

# Wireframe

Figure 13dashbord wireframe

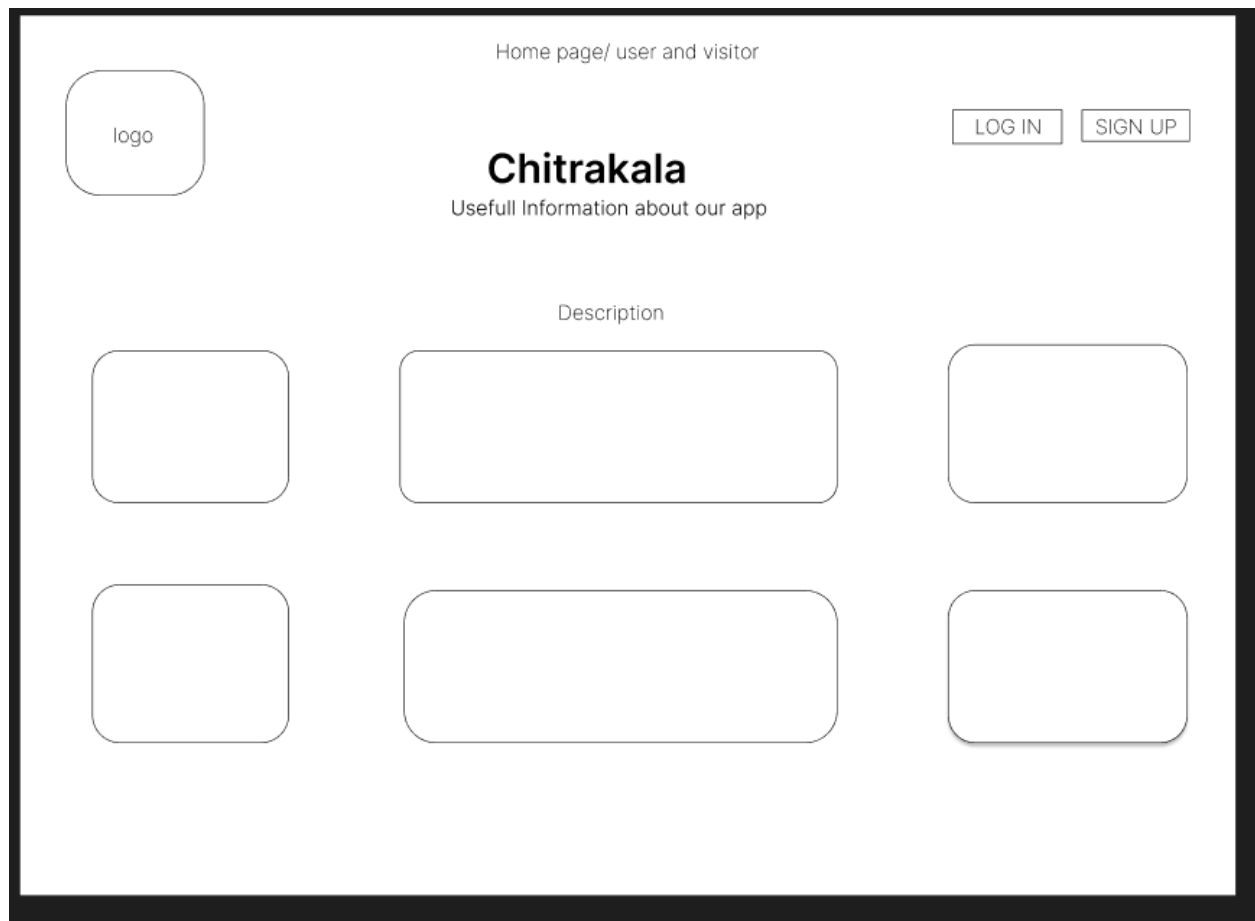


Figure 14login/signing wireframe

The wireframe shows two side-by-side forms on a dark background. The left form is titled 'LOGIN' and contains fields for 'Email' and 'Password', a 'Remember me?' checkbox, a 'LOGIN' button, and a 'Forgot Password?' link. Below these are social media icons for Google+, Facebook, and LinkedIn, followed by the text 'Need an account? SIGN UP'. The right form is titled 'SIGN UP' and contains fields for 'Email' and 'Password', a 'SIGN UP' button, and the same social media icons, followed by the text 'Already a user ? LOGIN'.

**LOGIN**

Email

Password

☒ Remember me?

LOGIN

Forgot Password?

Need an account? SIGN UP

**SIGN UP**

Email

Password

SIGN UP

Already a user ? LOGIN

Figure 15userinfo wireframe

The wireframe shows a 'Desktop - 2' view of a form. At the top is a 'Send Message' button. Below it is a large rounded rectangle titled 'Your Information'. Inside this rectangle are labels for 'Your Name', 'Email', 'Subject', and 'Any Message'. The 'Any Message' label is above a large text area. A 'Send' button is located at the bottom right of the 'Your Information' box.

Desktop - 2

Send Message

**Your Information**

Your Name

Email

Subject

Any Message

Send



Figure 16home page

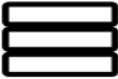
Wireframe - 1

## Find Your Art

location

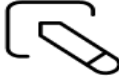
Price Range

Categories



FOR RESTUDENT

FOR SHOPPING MALL





FOR OFFICES

FOR FLAT/ROOM



Figure 17admin

Wireframe - 2

## Buy or selling

**Login as Artis**

**Login as User**

**Login as Admin**

New In page ? login now

## 9. Conclusion

The Chitrakala Digital Art Gallery integrates multiple design perspectives—functional decomposition, use cases, data models, component architecture, temporal sequences, and activity flows—into a coherent, implementable system design. Each diagram serves a specific communicative purpose: the FDD establishes system scope and organizational hierarchy, the use case diagram clarifies user interactions and system capabilities, the ERD ensures data integrity and minimal redundancy, the component diagram guides implementation modularity, sequence diagrams document temporal dependencies and integration points, and activity diagrams detail control flow and decision logic.

Together, these artifacts constitute a comprehensive blueprint enabling developers to implement the system with minimal ambiguity, enabling testers to verify all specified behaviors, enabling project managers to estimate scope and schedule, and enabling stakeholders to validate that the design addresses their requirements. The investment in rigorous system design substantially reduces downstream implementation risks and change costs.